

Глубокое обучение

Методы искусственного интеллекта

Лекция 11

Развитие органов зрения

550 миллионов лет назад число видов животных резко увеличилось.

Данный всплеск объясняется появлением светочувствительных клеток у трилобитов (зачатки органов зрения).

Появление зрения привело к перестройке всей пищевой цепочки и эволюции многих живых организмов.

У современных млекопитающих для организации органов зрения используется значительная часть коры головного мозга (внешнее серое вещество).

Изучение обработки визуальной информации

В конце 1950-х годов Дэвид Хьюбел и Торстен Визель провели исследования для изучения способов обработки визуальной информации в коре головного мозга млекопитающих (кошек).

За это исследование ученые получили Нобелевскую премию по физиологии и медицине в 1981 году.

В процессе исследования ученые показывали кошкам изображения и отслеживали активность отдельных нейронов первичной зрительной коры, которая получает импульсы от глаз.

Ученые обнаружили, что нейроны реагируют на смену слайда, а не на содержимое слайда.

При смене слайда по экрану пробегала черная полоса.

Обработка сигналов мозгом

Дэвид Хьюбел и Торстен Визель обнаружили, что нейроны, которые получают сигналы от глаза, чувствительны к краям объекта (простым линиям).

Такие нейроны были названы простыми.

Простые нейроны по разному реагируют на линии (края) с определенной ориентацией.

Отдельные группы нейронов «обрабатывают» отдельные ориентации линий.

Простые нейроны передают данные сложным нейронам.

Сложные нейроны объединяют линии в более сложные фигуры: угол, кривая и т. д.

Следующие нейроны формируют более сложные фигуры до получения конечного результата.

Компьютерное зрение

Устройство зрительной системы стало основой для современных глубоких методов машинного обучения.

Основные этапы развития систем компьютерного зрения:

1. Неокогнитрон. В конце 1970-х годов инженер Кунихико Фукусима предложил архитектуру для организации компьютерного зрения. Данная работа была основана на открытии простых и сложных нейронов. Неокогнитрон мог распознавать рукописные символы.
2. LeNet-5. В конце 1990-х годов исследователи Ян Лекун и Йошуа Бенжио предложили иерархическую архитектуру LeNet-5, которая была основана на работах Хьюбела, Визеля и Фукусимы. LeNet-5 удалось достичь таких результатов из-за наличия качественного набора данных рукописных цифр MNIST, мощной вычислительной техники и метода обратного распространения ошибки. LeNet-5 – первая глубокая модель, которую использовали в коммерческих целях.

Развитие компьютерного зрения

3. ImageNet. В конце 2000-х годов научная группа Фей-Фей Ли представила размеченный набор изображений ImageNet для машинного обучения моделей в области компьютерного зрения. На тот момент ImageNet содержал 14 миллионов изображений и 22000 классов.
4. ILSVRC. В 2010 году Фей-Фей Ли организовала конкурс ImageNet Large Scale Visual Recognition Challenge для моделей машинного обучения на основе ImageNet.
5. AlexNet. В 2012 году глубокая архитектура AlexNet разгромила традиционные модели в конкурсе ILSVRC. Архитектура AlexNet была разработана Алексом Крижевским и Ильей Суцкевером из лаборатории Университета Торонто под руководством Джеффри Хинтона. AlexNet удалось достичь таких результатов из-за наличия качественного набора данных ImageNet, мощной вычислительной техники (GPU) и новых подходов к построению глубоких нейронных сетей.

Традиционное и глубокое обучение

При использовании традиционных методов машинного обучения большая часть времени затрачивается на конструирование признаков.

Традиционные методы машинного обучения показывают хорошие результаты при поиске линейных зависимостей.

При использовании глубокого обучения акцент смещается на выбор и проектирование подходящей архитектуры глубокой нейронной сети.

Нейронные сети являются универсальными аппроксиматорами и показывают хорошие результаты при поиске линейных и нелинейных зависимостей.

Глубокие нейронные сети автоматически преобразуют входные данные в признаки по аналогии с мозгом.

Глубокое обучение в NLP

Основные этапы развития глубокого обучения для NLP:

1. 2012. AlexNet.
2. 2012-2014. Развитие глубокого обучения для компьютерного зрения.
3. 2015. Глубокие модели повышают качество в соревнованиях по NLP.
4. 2016-2017. Глубокие модели побеждают в соревнованиях по NLP.

Недостатки Text Mining:

1. Учитывается только статистика наличия и распределения слов в документе.
2. Отсутствует учет смысла и контекста слова.

Векторное представление слов

Алгоритм формирования эмбедингов для слов:

1. Выбирается целевое слово.
2. Для каждого целевого слова рассматривается его контекст (n слов слева и справа от целевого).
3. Для целевого слова вычисляется позиция в n -мерном пространстве с учетом контекста.

Схожие по смыслу с учетом контекста целевые слова будут находиться рядом в пространстве.

Популярные методы получения векторного представления слов:

- word2vec,
- GloVe,
- FastText, и др.

Векторная арифметика

Основным преимуществом векторного представления слов является возможность выполнения арифметических операций над векторами для поиска точек в n-мерном пространстве.

$$v_{\text{King}} = [-1.8, 3.8, 4.4]$$

$$v_{\text{Man}} = [-2.2, 4.8, 6.0]$$

$$v_{\text{Woman}} = [-6.4, 5.0, 5.2]$$

$$v_{\text{Queen}} = v_{\text{King}} - v_{\text{Man}} + v_{\text{Woman}}$$

$$v_{\text{Queen}} = \begin{bmatrix} -1.8 + 2.2 - 6.4 \\ 3.8 - 4.8 + 5.0 \\ 4.4 - 6.0 + 5.2 \end{bmatrix} = [-6.0, 4.0, 3.6]$$

<https://lamyowce.github.io/word2viz/> – пример работы с n-мерным пространством.

Векторная арифметика для изображений

Глубокие модели также способны формировать n-мерные пространства признаков для изображений и формировать векторные представления изображений.

К векторным представлениям изображений также применима векторная арифметика.



Генеративно-сопязательные сети

Генеративно-сопязательные сети (Generative Adversarial Network, GAN) – архитектура глубокой нейронной сети, основной задачей которой является генерация нового контента.

В GAN используются сверточные слои.

Архитектура GAN была разработана Яном Гудфеллоу в 2014 году (в тот момент был аспирантом Йошуа Бенжио (разработчик LeNet-5)).

GAN состоит из двух сетей:

1. Генератор. Генерирует контент.
2. Дискриминатор. Сравнивает «подлинный» и сгенерированный контент и пытается выявить «подделку».

В процессе обучения генератор учится генерировать контент лучшего качества, а дискриминатор лучше отличать подделку от оригинала.

«Подлинный» контент позволяет задать образец контента для генерации.

История разработки GAN

Друзья Гудфеллоу работали над моделью для генерации изображений на основе традиционных методов машинного обучения.

Основная сложность заключалась в конструировании признаков для обучения модели правильно располагать части объектов при генерации.

Результаты модели не были качественными:

- модель могла получить размытое изображение,
- части объекта могли располагаться неправильно,
- обязательные части объекта могли отсутствовать.

После обсуждения данной проблемы, Гудфеллоу вернулся домой и за ночь создал архитектуру GAN.

Первые результаты GAN

Результаты Гудфеллоу и коллег, описанные в статье 2014 года.



a)



b)



Передача стиля в GAN

В 2017 году Чжу, Пак и другие исследователи предложили новую архитектуру CycleGAN.

CycleGAN сохраняет целостность изображения в течение множества циклов обучения и позволяет передавать визуальный стиль.



Создание изображений из текста

В 2017 году была предложена архитектура StackGAN.

StackGAN состоит из двух GAN сетей: первая генерирует вход для второй.

Первая сеть генерирует грубое изображение в плохом качестве на основе текстового описания.

Вторая сеть улучшает качество изображения.

Птица с желтым брюхом и лапками, серой спинкой и крыльями, с коричневым горлом и затылком и с черным клювом

Белая птица с черным пятнами на голове и крыльях и длинным оранжевым клювом

Цветок с розовыми заостренными лепестками, накладывающимися друг на друга и окружающими сердцевину с короткими желтыми волосками

(a) Изображения, полученные на первом этапе



(b) Изображения, полученные на втором этапе



Обработка изображений

В 2018 году Чен Чен и его коллеги предложили архитектуру U-Net для обработки изображений.

В настоящий момент глубокие модели используются для обработки изображений в составе специализированного программного обеспечения, в том числе в приложениях смартфонов.



Глубокое обучение с подкреплением

Основные успехи в глубоком обучении с подкреплением показывают различные проекты DeepMind в области игр.

DeepMind – британский технологический стартап, основанный Демисом Хассабисом, Шейном Леггом и Мустафой Сулейманом в 2010 году.

В 2014 году компания Google приобрела DeepMind за 0.5 млрд долларов США.

Основные успехи DeepMind:

1. 2013-2015. DQN (Deep Q-Learning) – модель для прохождения 49 игр Atari 2600 с нуля на основе анализа изображений с экрана.
2. 2016. AlphaGo – модель для игры в Go. Модель победила Ли Седола (мировой чемпион) со счетом 4:1. Первоначальное обучение с учителем.
3. 2016. AlphaGo Zero. В отличие от AlphaGo обучалась с нуля. За три дня модель сыграла почти 5 млн. игр сама с собой. За 36 часов обучения опередила AlphaGo. Модель открыла новые стратегии игры.
4. 2017. AlphaZero. Модель для игры в Шахматы, Сёги и Go. Одна модель обучалась играть во все игры с нуля. За 700 тыс. шагов обучения модель стала лидером во всех играх.

Нейрон

Искусственные нейроны были созданы по аналогии с биологическими.

Каждый нейрон получает входные сигналы от множества дендритов.

Нейроны соединены между собой дендритами и образуют нейронную сеть.

Сигнал от одних дендритов повышает электрический потенциал нейрона, от других уменьшает его.

Если электрический потенциал нейрона достигает некоторого порога (потенциал действия), то нейрон посылает сигнал другим нейронам по аксону.

У каждого нейрона существуют:

1. Дендриты (входы).
2. Аксон (выход).
3. Потенциал действия (функция активации).

Перцептрон

В 1958 году Фрэнк Розенблатт предложил концепцию перцептрона.

Перцептрон считается первой попыткой создания искусственного нейрона на основе понимания принципов работы биологических нейронов.

Перцептрон содержит:

1. Множество бинарных входов (X).
2. Бинарный выход (y).
3. Пороговую функцию активации:

$$f(X) = \sum_{i=1}^n w_i x_i$$
$$y = \begin{cases} 1, & f(X) > k \\ 0, & f(X) \leq k \end{cases}$$

Уравнение нейрона

Принцип работы любого нейрона можно представить в виде уравнения нейрона:

$$\begin{aligned}f(x) &= w \cdot x + b, \\w \cdot x &= \sum_{i=1}^n w_i x_i, \\b &= -k, \\y &= \begin{cases} 1, & f(x) > 0 \\ 0, & otherwise \end{cases}\end{aligned}$$

$W, w_i \in W$ – вектор коэффициентов, полученных в процессе обучения.

$X, x_i \in X$ – вектор входных признаков.

b – смещение (bias) как инверсия порогового значения k перцептрона.

Функции активации

1. Пороговая.
2. sigmoid (сигмоида)

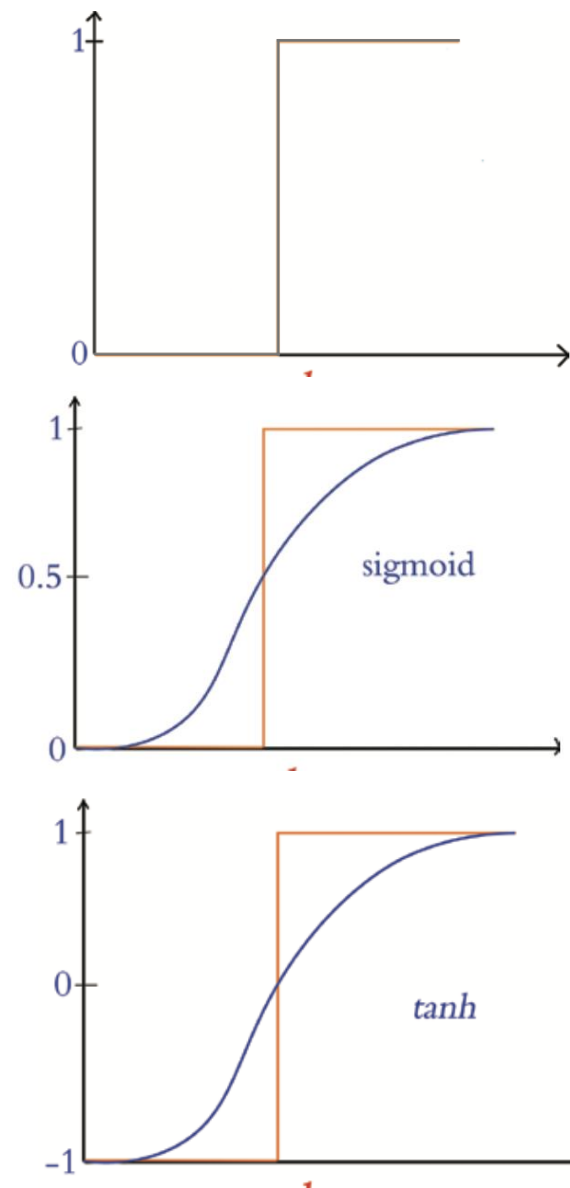
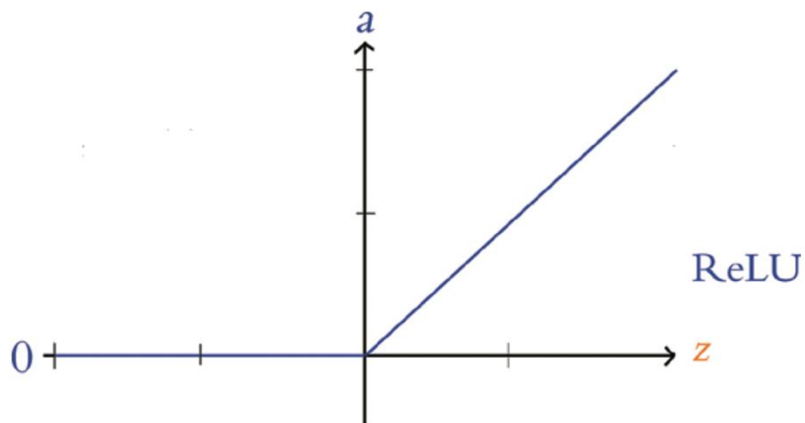
$$y = \frac{1}{1 + e^{f(x)}}$$

3. Tanh (танх)

$$y = \frac{e^{f(x)} - e^{-f(x)}}{e^{f(x)} + e^{-f(x)}}$$

4. ReLU (Rectified Liner Unit, блок линейной ректификации)

$$y = \max(0, f(x))$$



Выбор типа нейронов

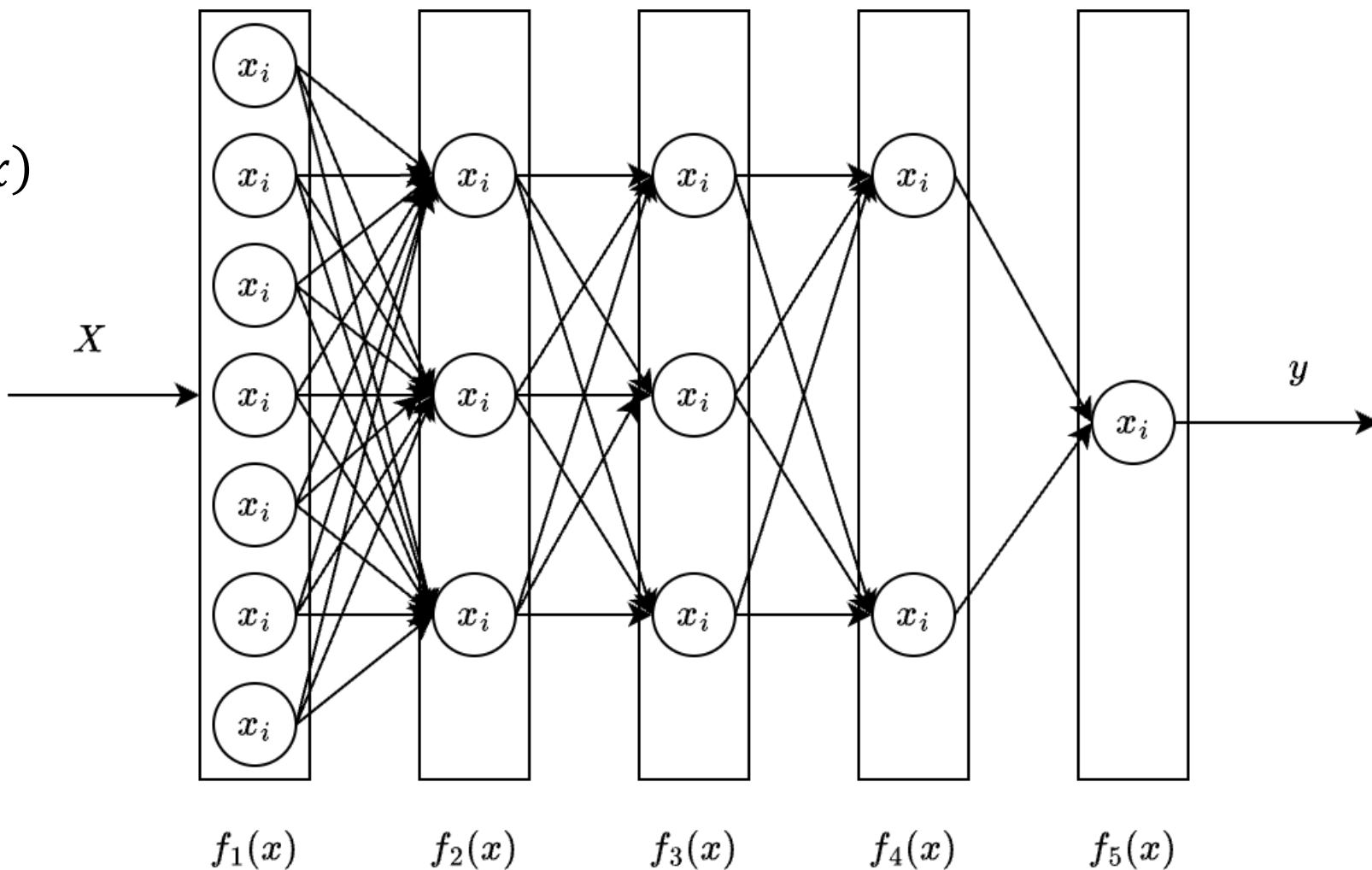
1. Перцептрон содержит бинарные входы и выход, а также использует пороговую функцию активации. Не подходит для глубокого обучения.
2. Сигмоидный нейрон можно использовать для глубокого обучения, но такие нейроны работают медленнее, чем $\tanh$ и ReLU нейроны. Обычно используются для слоев с выходом, значение которого изменяется в диапазоне $[0,1]$.
3. $\tanh$ -нейрон хороший выбор для глубоких нейронных сетей. Выходное значение изменяется от $[-1,1]$.
4. ReLU-нейрон на данный момент показывает более высокую эффективность обучения: скорость и точность обучения.

Структура нейронной сети

Входной слой $f_1(x)$

Скрытые слои $f_2(x), f_3(x), f_4(x)$

Выходной слой $f_5(x)$



Входной и выходной слои

Входной слой принимает на вход вектор признаков. Количество нейронов во входном слое должно соответствовать количеству признаков.

Выходной слой формирует конечный результат. Количество нейронов в выходном слое определяется задачей машинного обучения: классификация, регрессия, кластеризация.

Для классификации можно использовать следующие функции активации:

1. sigmoid для бинарной классификации.
2. softmax для многоклассовой классификации:

$$y_i = \exp(x_i) / \sum_{i=1}^n \exp(x_i)$$

Скрытые слои

Полносвязные слои (плотные слои, dense layers) – слой нейронной сети, в котором все нейроны текущего слоя связаны с каждым нейроном предыдущего слоя (каждый нейрон имеет полный набор связей с предыдущим слоем).

Полносвязные слои позволяют нелинейно комбинировать информацию от предыдущего слоя.

Взаимодействие между слоями искусственной нейронной сети происходит на основе прямого распространения сигнала.

Каждый нейрон полносвязного слоя можно представить как:

$$f(x) = \sum_{i=1}^n w_i x_i + b$$
$$y = a(f(x))$$

n – количество нейронов в предыдущем слое;

a – функция агрегации (sigmoid, tanh, ReLU, softmax, и т. д.).

Основные архитектуры для разных задач

Для решения различных задач обучения и анализа данных были разработаны различные архитектуры глубоких нейронных сетей.

Компьютерное зрение:

- Сверточные нейронные сети (Convolutional neural network, ConvNet, CNN).

NLP:

- Рекуррентные нейронные сети (Recurrent Neural Network, RNN).
- Ячейки с долгой краткосрочной памятью (Long Short-Term Memory, LSTM). LSTM входит в семейство RNN.

RNN можно применять для обработки любых данных, имеющих последовательную форму, например, временные ряды.

Функция стоимости

Процесс обучения ИНС можно представить как $C(y, \hat{y}) \rightarrow \min$

C – функция стоимости;

$y, \hat{y}$ – реальные и предсказанные значения целевого признака.

Квадратичная функция (среднеквадратичная ошибка, mse) используется в задачах регрессии

$$C = \frac{1}{n} \sum_{i=1}^n (y - \hat{y})^2$$

Возведение в квадрат гарантирует отсутствие отрицательных значений и увеличивает штраф за большую ошибку (разницу между фактом и предсказанием).

В процессе обучения происходит подбор w и b всех нейронов ИНС для минимизации значения функции потерь.

В некоторых случаях изменение w и b слабо влияет на изменение значения функции потерь и обучение замедляется, тогда говорят о насыщении нейрона.

Перекрестная энтропия

Для решения проблемы насыщения нейронов часто используется функция потерь на основе перекрестной энтропии:

$$C = \frac{1}{n} \sum_{i=1}^n [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)]$$

Свойства перекрестной энтропии:

1. Линейное увеличение разницы между y и $\hat{y}$ приводит к экспоненциальному увеличению стоимости.
2. Чем выше ошибка, тем быстрее происходит обучение. Используется вычисление частной производной. Перекрестная энтропия структурирована для облегчения вычисления частной производной.

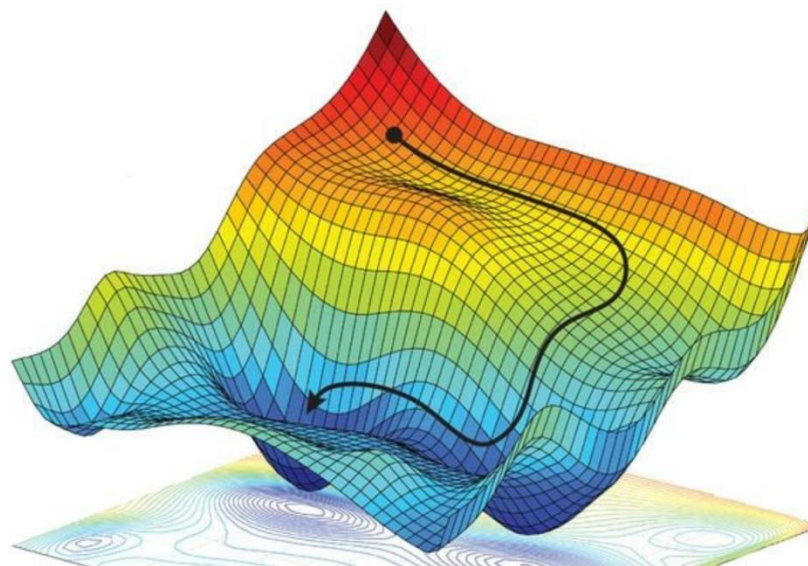
Градиентный спуск

Градиентный спуск – подход для настройки параметров модели с целью минимизации стоимости.

Градиентный спуск широко используется в машинном обучении.

При градиентном спуске оценивается некоторая окрестность точки в n -мерном пространстве для нахождения следующей точки с меньшим значением функции стоимости.

В процессе обучения градиентный спуск вычисляет наклон поверхности (через нахождение частных производных) и корректирует параметры нейрона, чтобы получить наименьшее значение функции стоимости.



Скорость обучения

Скорость обучения (η) – гиперпараметр модели ИНС, отвечающий за размер шага в n -мерном пространстве при использовании градиентного спуска в процессе обучения.

Важно:

- Чем меньше значение скорости обучения, тем больше шагов обучения необходимо сделать.
- Чем больше значение скорости обучения, тем меньше вероятность найти глобальный минимум (в процессе обучения его можно «перешагнуть»).

Стохастический градиентный спуск

Проблемы классического градиентного спуска:

1. Не во всех случаях можно поместить весь набор данных в ОЗУ.
2. Большие временные затраты (высокая вычислительная сложность) на обучение модели с большим числом параметров.

Данные проблемы решаются с помощью стохастического градиентного спуска (Stochastic Gradient Descent, SGD).

При использовании SGD данные делятся на пакеты некоторого размера (batch size).

$$\text{количество пакетов} = \frac{\text{размер набора данных}}{\text{размер пакета}}$$

Важно:

- Перед разделением данных на пакеты строки перемешиваются случайным образом.
- Последний пакет может иметь размер меньше заданного.

Процесс обучения

1. Разделение данных на пакеты.
2. Выборка i -го пакета с n образцами данных (x).
3. Прямое распространение x через сеть и получение $\hat{y}$.
4. Вычисление значения функции стоимости C .
5. Градиентный спуск для минимизации C на основе корректировки параметров w и b нейронов. Минимизация применяется для получения более точного прогноза $\hat{y}$.
6. Если есть следующий пакет, то перейти к шагу 2, иначе закончить.

Важно:

- В процессе обучения данные в пакетах не повторяются, а пакеты не перемешиваются.
- После завершения процесса обучения происходит переход к следующей эпохе обучения. Обучение завершается при достижении установленного значения количества эпох.
- При начале новой эпохи происходит формирование новых пакетов со случайными данными.

Ловушка локального минимума

При градиентном спуске можно попасть в локальный минимум и не достигнуть глобального минимума (наименьшее достижимое значение функции стоимости).

SGD, при правильном подборе размера пакетов, может сглаживать пространство поиска и устраняет проблему с ловушкой локального минимума.

Важно:

- При использовании пакетов большого размера оценка градиента функции стоимости будет точнее за счет исследования большего пространства вокруг текущей точки, но существует возможность попасть в ловушку локального минимума.
- При использовании большого размера пакета требуется больше памяти и больше времени на обучение.
- Если размер ловушки меньше скорости обучения, то возможно «перешагнуть» через ловушку.
- При использовании маленьких пакетов снижается качество оценки градиентного спуска за счет сокращения видимого пространства.
- При использовании маленьких пакетов повышается время обучения за счет увеличения длины пути градиентного спуска, а вычислительные ресурсы будут использованы не полностью.

Обратное распространение ошибки

Обратное распространение ошибки используется для передачи значения функции стоимости от выходного слоя ко входному слою через скрытые слои глубокой модели.

Обратное распространение ошибки позволяет регулировать параметры скрытых слоев.

Прямое распространение: $x \rightarrow$

Обратное распространение: $\leftarrow C$

Важно:

- Происходит вычисление вклада отдельного параметра (градиент) в общую стоимость с последующей корректировкой значения данного параметра для уменьшения стоимости.
- Обратное распространение сильнее влияет на слои ближе к выходному и слабо на слои ближе ко входному слою.

Нестабильность градиента

Исчезающие градиенты – чем больше в ИНС слоев, тем хуже работает обратное распространение ошибки (градиент параметра выравнивается по стоимости при движении ко входному слою).

Взрывные градиенты – увеличение крутизны угла градиента параметра по стоимости при движении ко входному слою.

Для решения проблем неустойчивости градиента обычно используют пакетную нормализацию.

При пакетной нормализации из выходных данных предыдущего слоя вычитается среднее значение по пакету, результат делится на стандартное отклонение в пакете.

Свойства пакетной нормализации:

- возможность для слоев обучаться независимо друг от друга, слои не влияют друг на друга;
- более высокая скорость обучения за счет отсутствия экстремальных значений;
- нормализация осуществляется на основе данных пакета, что добавляет шум, который способствует регуляризации.

Переобучение

При переобучении ИНС «запоминает» обучающий набор и стоимость близкую к нулю, но на тестовой выборке стоимость растет на каждой следующей эпохе.

Методы борьбы с переобучением:

1. L1 и L2 регуляризация. Используются в традиционном машинном обучении. Основаны на добавлении суммы абсолютных значений коэффициентов параметров в функцию стоимости. Чем больше величина параметра, тем больший вклад параметр вносит в функцию стоимости. Из модели исключаются незначимые параметры, модель становится проще.
2. Прореживание (dropout). Более подходящий для глубоких ИНС метод регуляризации. При использовании прореживания на каждом цикле обучения происходит отключение некоторого количества нейронов в указанных скрытых слоях. При использовании прореживания входные данные необходимо умножать на 1 – количество выключенных нейронов (0.5 для 50 %, 0.667 для 33 %). Некоторые библиотеки выполняют это действие автоматически.

Настройка гиперпараметров ИНС

Скорость обучения:

1. Начальное значение: 0.01 или 0.001.
2. Если стоимость будет медленно уменьшаться с каждой эпохой (итерацией) обучения, то можно увеличить стоимость обучения на порядок (с 0.01 до 0.1).
3. Если стоимость хаотично увеличивается и уменьшается, то необходим уменьшить скорость обучения на порядок (с 0.01 до 0.001).

Размер пакета:

1. Начальное значение: 32.
2. Если пакет не помещается в ОЗУ, то следует уменьшить размер на одну степень двойки (с 32 до 16).
3. Если стоимость с каждой эпохой снижается, но обучение требует много времени, то следует увеличить размер пакета на степень двойки (с 32 до 64).
4. Не рекомендуется использовать размер пакета больше 128.

Количество эпох:

1. Если стоимость на тестовых данных снижается и последняя эпоха имеет наименьшую стоимость, то можно увеличить число эпох.
2. Если стоимость на тестовых данных начинает расти, то произошло переобучение модели и следует уменьшить число эпох.
3. Можно использовать методы оценки стоимости для автоматической остановки обучения.

Еще настройка гиперпараметров ИНС

Количество скрытых слоев:

1. Рекомендуется начинать с 2-4 скрытых слоев.
 - Чем больше слоев, тем более абстрактные признаки способна находить ИНС в данных.
 - Чем больше слоев, тем менее эффективно обратное распространение ошибки.
2. Если уменьшение количества слоев не увеличивает стоимость на тестовой выборке, то рекомендуется уменьшить количество скрытых слоев.
 - Чем меньше слоев, тем быстрее обучается ИНС, а обучение требует меньше ресурсов.
3. Если увеличение количества слоев снижает стоимость на тестовой выборке, то следует увеличить количество скрытых слоев.

Количество нейронов в слое:

1. Если количество нейронов в слое больше необходимого, то это незначительно увеличит вычислительную сложность.
2. Если количество нейронов в слое немного меньше необходимого, то это почти не скажется на качестве модели.

Снова настройка гиперпараметров ИНС

Прореживание слоев:

1. Если наблюдается переобучение, то рекомендуется применить прореживание.
2. Прореживание рекомендуется использовать даже если переобучение отсутствует.
3. Прореживание обычно применяется только к последним скрытым слоям. Для начала можно применить прореживание только к последнему слою. Если наблюдается переобучение, то можно добавить прореживание к предпоследнему скрытому слою.
4. Если сеть показывает высокую стоимость на тестовой выборке, то необходимо уменьшить долю отключаемых нейронов. Модель получилась слишком простой.
5. Для решения задач компьютерного зрения рекомендуется отключать от 20 до 50 % нейронов скрытого слоя. Для задач NLP от 20 до 30 %.

Гиперпараметры подбираются на основе опыта решения различных задач на основе различных архитектур глубоких ИНС.

Оптимизаторы

Основные оптимизаторы:

1. SGD.
2. Метод моментов.
3. Метод Нестерова.
4. AdaGrad.
5. AdaDelta.
6. RMSProp.
7. Adam.

Выводы и рекомендации

1. Глубокое обучение позволяет выявить более сложные нелинейные закономерности из данных.
2. При использовании глубокого обучения требуется меньше времени на процесс конструирования признаков.
3. Глубокое обучение в настоящий момент позволяет решать любые задачи машинного обучения для любых типов данных.
4. Глубокое обучение основывается на векторной арифметике.
5. Глубокие ИНС способны генерировать новые данные.
6. ИНС состоят из нейронов. Нейрон содержит входы, выход и функцию активации.
7. ИНС состоит из слоев: входной, выходной, скрытые слои.
8. Обучение ИНС основано на функции стоимости и градиентом спуске.
9. Стохастический градиентный спуск позволяет обучаться на больших объемах данных.
10. Обучение ИНС основано на прямом и обратном распространении.
11. Для получения качественной ИНС необходимо подобрать оптимальные гиперпараметры.
12. Оценить качество модели можно с помощью традиционных метрик качества.